

Freight Rate Prediction

Machine Learning Engineer Assessment

Candidate: **Ayush Dadhich** Repository: <https://github.com/ayushd64/freight-rate-ml>

1. Summary

A LightGBM model predicts posted freight rates with a mean absolute error of \$61.09, or 2.56% of the typical rate, measured across three rolling-origin validation folds. That is a 69% reduction against a constant rate-per-mile baseline and a 59% reduction against ridge regression on the same features.

Two findings drove most of the result. First, the labelled data contains roughly 1.4% of rows whose target has been multiplied or divided by a factor between two and six; excluding them from training is worth more than every feature engineering decision combined. Second, the task is a genuine forecast — training data ends on 31 October and predictions are required through 31 December — so the validation design has to reproduce that forward gap rather than shuffling rows at random.

2. Data and validation strategy

2.1 The data is a time series, not an i.i.d. sample

The labelled file covers 1 January to 31 October 2025 (48,000 loads). The validation file covers 1 November to 31 December 2025 (12,000 loads) with no overlapping dates. Although the instructions describe validation as November, it in fact spans both months.

This makes a random train/test split actively misleading. Splitting at random places loads from the same week on both sides of the divide, so the model is scored on interpolation within a period it has already seen. The real task is extrapolation two months beyond the end of the training window. A random split would report a score the model cannot reproduce on the data that actually counts.

2.2 Rolling-origin validation

Internal validation therefore uses expanding training windows with fixed two-month test blocks, so that every fold asks the same question the real task asks:

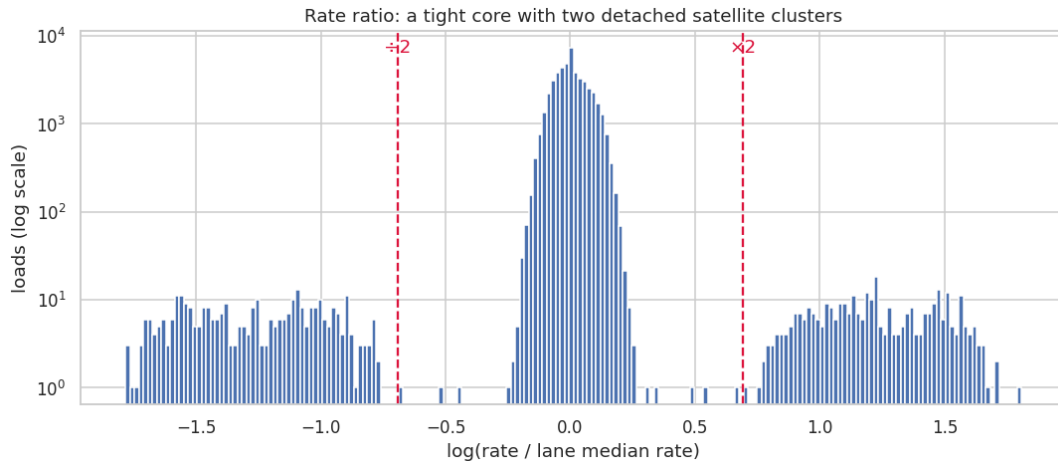
Fold	Trains on	Tests on	Test loads
1	Jan – Apr	May – Jun	9,696
2	Jan – Jun	Jul – Aug	9,671
3	Jan – Aug	Sep – Oct	9,523

Hyperparameters were selected on the mean across all three folds rather than on a single holdout. Fold-to-fold variation is substantial — fold 2 is consistently the hardest — and tuning against any one of them would have produced a model fitted to the quirks of one two-month window.

The final model then refits on the complete January to October window. Since the real prediction period runs to 31 December, holding data back at that stage would only weaken the model; the hyperparameters were already fixed by cross-validation, so nothing is being tuned on data the final model trains on.

2.3 Corrupted labels are excluded from training only

Comparing each load's rate per mile against the median for its own origin-destination lane controls for distance, geography and lane economics at once. The resulting ratio forms a tight unimodal core flanked by two clearly detached clusters, with an empty gap between them — the signature of injected noise rather than a heavy natural tail.



Rate relative to lane median. Horizontal axis is the log ratio, vertical axis is a log count. The core ends at $|\log \text{ratio}| \approx 0.25$; the satellite clusters do not begin until ≈ 0.75 .

The threshold was placed inside that observed gap — a factor of two — rather than at an arbitrary percentile. This matters: a factor of three, which looks like the obvious round number, cuts through the middle of the satellite clusters and leaves about a third of the corrupted rows in the training data.

672 rows (1.40%) are excluded from model fitting. They are never excluded from evaluation of the validation set, which requires a prediction for every row regardless of whether its label looks plausible. Validation metrics are reported on uncorrupted rows so that they measure modelling quality rather than the unpredictable corruption.

3. Data quality

Five issues were identified during exploration and are handled in the pipeline:

Issue	Train	Validation	Treatment
Negative weight	292	145	Absolute value
Missing weight	300	165	Median imputation
Missing market_index	374	249	Daily mean, then median
Corrupted posted_rate	672	n/a	Excluded from training
Cities unseen in training	—	8	Geography by coordinate

The negative weights are sign flips rather than corrupted magnitudes: their absolute values span the same 5,000–47,500 lb range as valid rows and price identically. Repairing rather than dropping matters because 145 of them are validation loads that still require predictions.

Eight cities appear only in validation — Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk and San Diego — touching 12% of validation rows. Any model that encodes city names as categories has no representation for these. Because every city maps to exactly one coordinate pair, geography is instead carried by latitude and longitude, which lets an unseen city inherit an estimate from its neighbours.

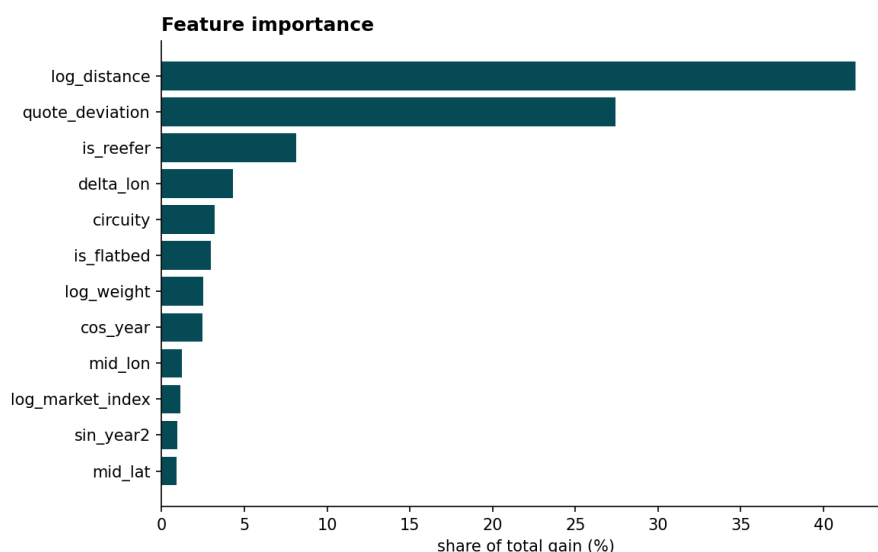
4. Features and model

The model predicts $\log(\text{rate per mile})$ and reconstructs the rate as $\exp(\text{prediction}) \times \text{distance}$. Rates follow a clean power law ($\text{rate} \approx \text{distance}^{0.874}$), so this representation bakes in the distance relationship, makes errors multiplicative — matching how freight prices behave — and stabilises variance across a target spanning two orders of magnitude.

Nineteen features. Three choices worth explaining:

- Coordinates rather than city names, for the unseen-city reason above. Verified by holding eight cities out of training entirely: coordinates scored \$30.90 MAE against \$34.78 for city IDs.
- Day-of-year sine and cosine harmonics for seasonality. Unlike a date ordinal these remain defined beyond the training window, so the fitted annual cycle carries forward instead of clamping at its last observed value.
- `quote_signal` recoded as absolute deviation from its median. The raw value has no monotonic relationship with price — only distance from the centre matters.

`market_index` is averaged to a daily figure before use. Within-day variation is only about 15% of its total variation, so it is fundamentally a market-wide daily quantity with per-load measurement noise on top. It carries the entire day-of-week effect: once it is in the model, weekday residuals are flat, so no separate weekday feature is needed.



Gain-based feature importance for the final LightGBM model.

5. Results

Means across the three rolling-origin folds, computed on rows with uncorrupted labels:

Model	MAE	RMSE	MAPE	Worst fold
Median \$/mile baseline	\$198.08	\$284.11	9.40%	\$203.69
Ridge regression	\$147.53	\$206.24	6.12%	\$238.75
LightGBM	\$61.09	\$112.40	2.56%	\$64.22

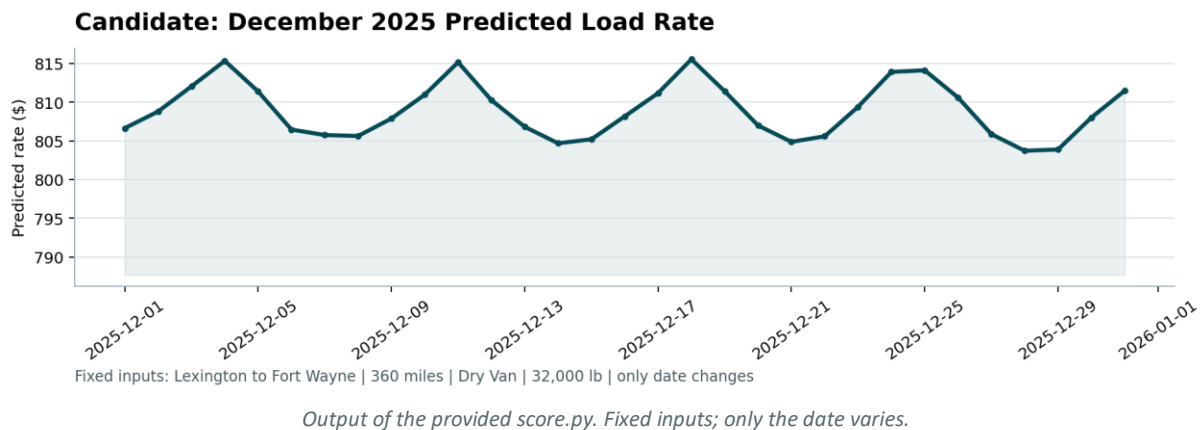
The assessment does not state which metric is used for final scoring, so MAE, RMSE, MAPE and median APE are all reported rather than optimising for one. MAE and MAPE reward the conditional median while RMSE rewards the conditional mean; with residual scatter this tight the two targets differ by roughly 0.1%, so the choice does not materially change the predictions.

Ridge is included not as a serious contender but as a calibration point. Its poor showing on fold 2 (\$238.75) against LightGBM's \$64.22 shows how much of the signal is nonlinear — principally the interaction between geography and haul length.

6. December prediction chart

The chart file supplies only seven columns: no market_index, no quote_signal, no coordinates. The model cannot score it directly.

The resolution is that validation.csv spans 1 November to 31 December and does carry market_index, at roughly 200 loads for every December day. Averaging it per date recovers the daily market level, which continues seamlessly from the training series with its weekly cycle intact. Coordinates for Lexington and Fort Wayne come from the city lookup built across both labelled files, and the quote signal — which does not exist for a hypothetical load — is set to the training median deviation, giving the typical load for each date.



The weekly sawtooth is the signal that the extrapolation worked. Five peaks appear, one per week, with weekdays averaging \$810.12 against \$805.65 at weekends — the market_index cycle carried into a period the model never saw. A flat line would have indicated a model unable to extrapolate, with every December day collapsing to a single value.

The predicted level, \$2.247 per mile, sits just below October's observed \$2.207 and well above January's \$2.068. The model therefore forecasts a mild seasonal decline into December rather than a full reversion to the winter trough.

7. Limitations

- Corruption detection relies on a lane median, which needs at least three loads on a lane to be meaningful. Roughly 300 of 48,000 rows sit on thinner lanes and cannot be checked this way.
- Residual scatter is about 5% and appears largely irreducible. The gap between a well-specified linear model and gradient boosting is real, but the remaining headroom above LightGBM is small.
- The training data covers ten months, so the annual seasonal cycle is never observed in full. The November–December seasonal component is an extrapolation of a curve fitted to January–October and cannot be validated against held-out data.
- No external data was used. Fuel prices, capacity indices and holiday calendars would plausibly explain part of the residual, particularly around late December.

Appendix A. Feature ablations

Each row removes one component and refits, with all else held constant. Mean MAE across the three rolling-origin folds.

Removed	MAE	Cost
Nothing (final model)	\$61.09	—
quote_deviation	\$71.96	+\$10.87
log_market_index	\$67.52	+\$6.43
Seasonal harmonics	\$65.30	+\$4.21
Corruption filter disabled	\$84.32	+\$23.23

Excluding corrupted rows is worth more than every feature decision combined — the single most consequential result in the project.

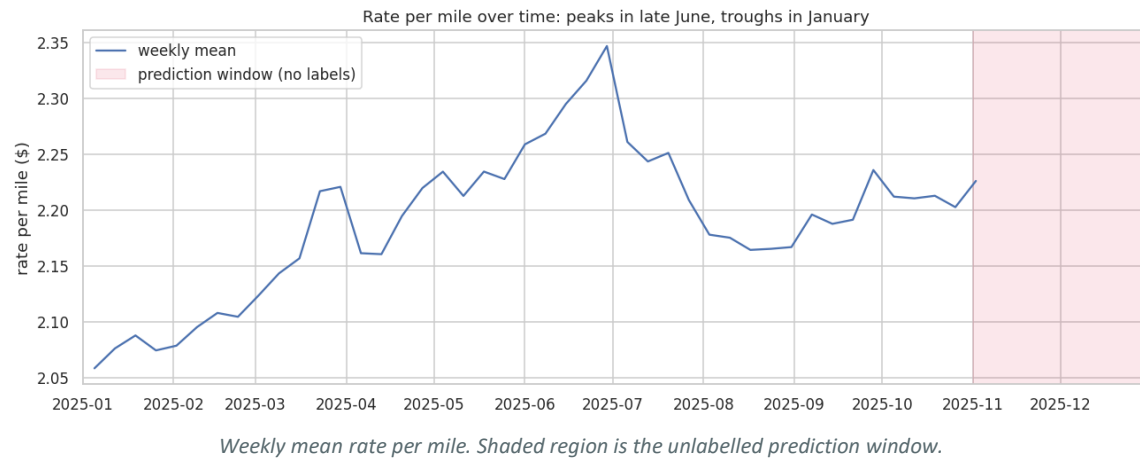
quote_deviation is worth recording as a methodological note. Univariate binning suggested a modest effect confined to the tails of its range, yet ablation shows it is the second most valuable feature after distance. Its contribution is conditional on the other features and is invisible to marginal analysis — a reminder that feature selection based on univariate screening would have discarded it.

Appendix B. Seasonality encoding

Three encodings were compared across the same folds: day-of-year harmonics (\$62.15 MAE), a raw date ordinal (\$60.98) and no time feature at all (\$66.93). Harmonics and the date ordinal are statistically indistinguishable on this evidence, and all three place the December level within 2% of each other.

The informative pattern is in the per-fold breakdown. Without any seasonality feature, the furthest-extrapolating fold degrades most sharply — from \$62.29 to \$85.42 — while the nearest fold barely moves. Seasonality matters more the further ahead the prediction reaches, which is precisely the regime December occupies.

Harmonics were chosen over the date ordinal on principle rather than measured performance: beyond the training range a harmonic is defined by the fitted annual cycle, whereas a tree given a date ordinal clamps to whatever it learned for late October. With the chart's entire purpose being variation across dates, the encoding whose out-of-range behaviour is intentional is the safer choice.



Appendix C. Reproduction

From the repository root, with the provided CSV files placed in data/:

```
pip install -r requirements.txt
python -m src.model          # validation metrics
python -m src.predict        # writes both submission files
python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv
```

All random seeds are fixed at 42. `src/predict.py` retrains from scratch rather than loading a serialised model, so the committed predictions cannot drift from the committed code. `requirements-lock.txt` records the exact package versions these results were produced with.